



Desarrollo con Spring Boot

```
01 @SpringBootApplication
02 public class Aplicacion {
03     public static void main(String[] args) {
04         SpringApplication.run(Aplicacion.class, args);
05     }
06 }
07
08 @RestController
09 @RequestMapping("/api")
10 public class SaludoController {
11     @GetMapping("/saludo")
12     public String saludo() {
13         return "¡Hola desde Spring Boot!";
14     }
15 }
```



IoC



MVC



REST



JPA



IoC. Inyección de dependencias. Beans.
MVC. REST APIs. Spring Data. Seguridad.



INYECCIÓN DE
DEPENDENCIAS



SPRING MVC

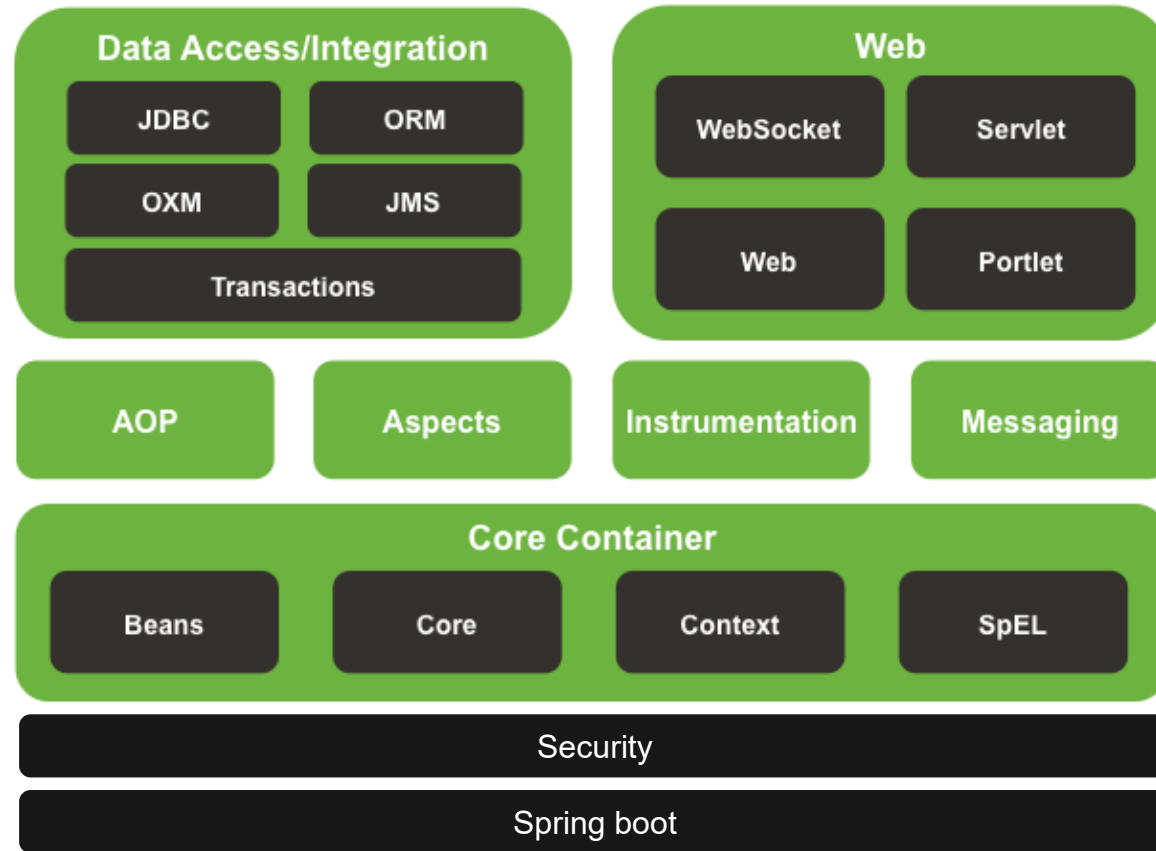


REST APIs



SPRING DATA JPA

SPRING FRAMEWORK



Spring Framework: módulos principales

Spring Framework ofrece los módulos base. Spring Boot simplifica su uso mediante autoconfiguración, starters y configuración externalizada

spring-core	Contenedor IoC, beans, inyección de dependencias
spring-context	ApplicationContext, eventos, internacionalización, configuración
spring-beans	Definición, creación y ciclo de vida de beans
spring-aop	Programación orientada a aspectos
spring-expression	SpEL, expresiones dinámicas
spring-jdbc	Acceso JDBC simplificado
spring-tx	Gestión declarativa de transacciones
spring-orm	Integración con JPA/Hibernate
spring-web	Infraestructura web
spring-webmvc	Modelo MVC clásico
spring-test	Testing unitario e integración



Tipos de Aplicaciones

Spring permite la creación de todo tipo de aplicaciones: aplicaciones web, aplicaciones multicapa, clientes ricos.

- **No es invasivo:** permite su uso conjunto a otros frameworks (p.e.: Struts o Hibernate).
- **No es todo o nada:** no obliga a la implementación de toda la aplicación empleando Spring.
- **Aplicaciones empresariales “ligeras” y basadas en Java EE:** no requieren de un servidor de aplicaciones, solamente una máquina virtual. Pero también se pueden integrar con servidores JEE para obtener las ventajas que estos ofrecen (Seguridad frente a fallos, escalabilidad, etc).
- **Sistema declarativo de manejo de transacciones:** basado en AOP puede manejar de forma declarativa las de un único recurso transaccional (p.e.: un DataSource). Si se desean realizar transacciones globales (que agrupen varios recursos transaccionales) se deberá usar una transacción de un servidor JEE mediante JTA.

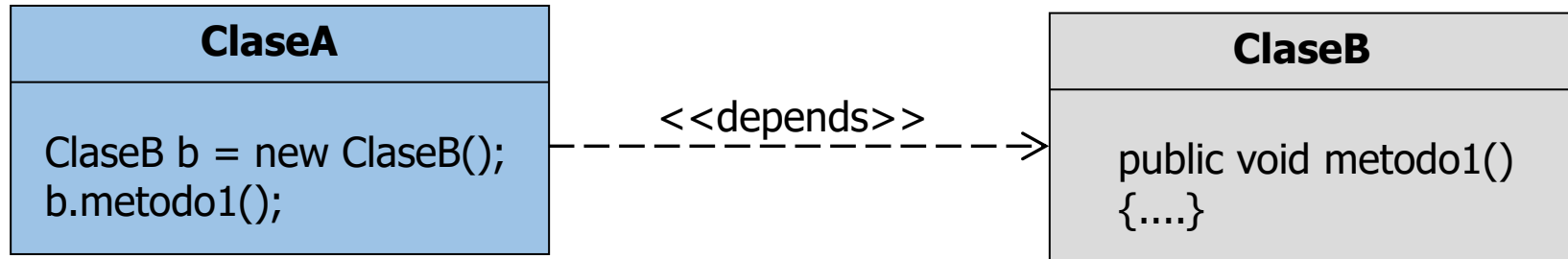
SECCIÓN

Dependencias

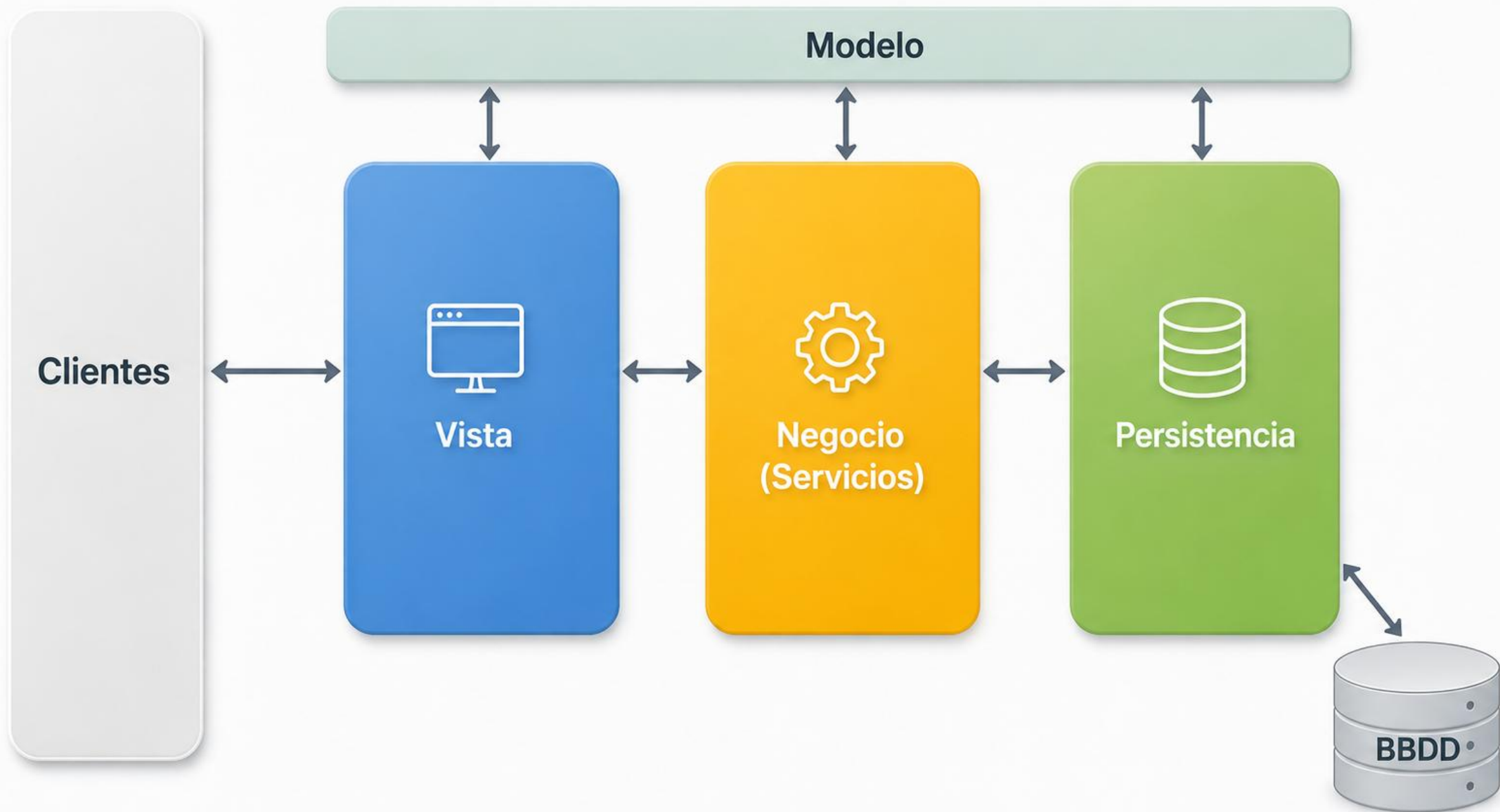


Dependencias

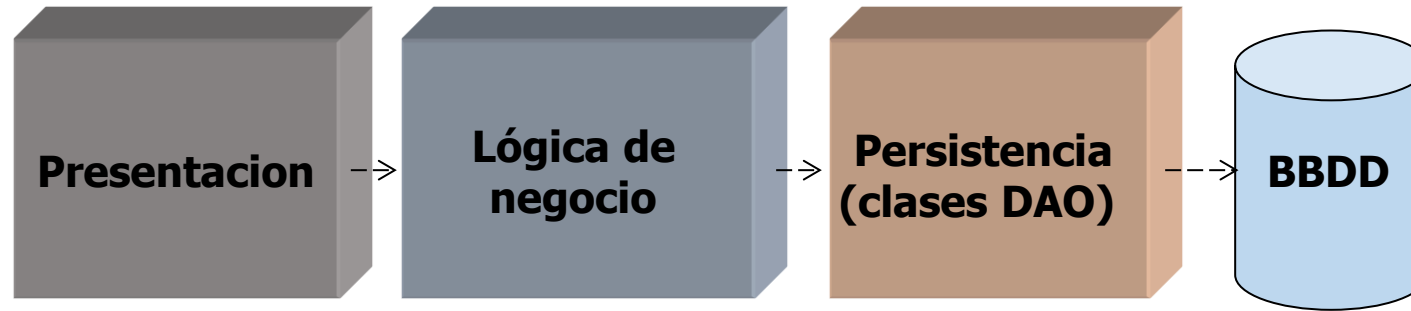
Se dice que una clase A depende de otra B cuando A tiene referencias de tipo B y crea objetos de B



- 1.- Difícil mantenimiento y rediseño: En tiempo de diseño, cualquier cambio realizado en una clase, puede afectar a las clases que dependen de ella y por lo tanto que sea necesario modificarlas también.
- 2.- Menor estabilidad: En tiempo de ejecución, si una parte de la aplicación tiene un error o deja de funcionar, si el sistema tiene muchas dependencias, se verá mayormente afectado.

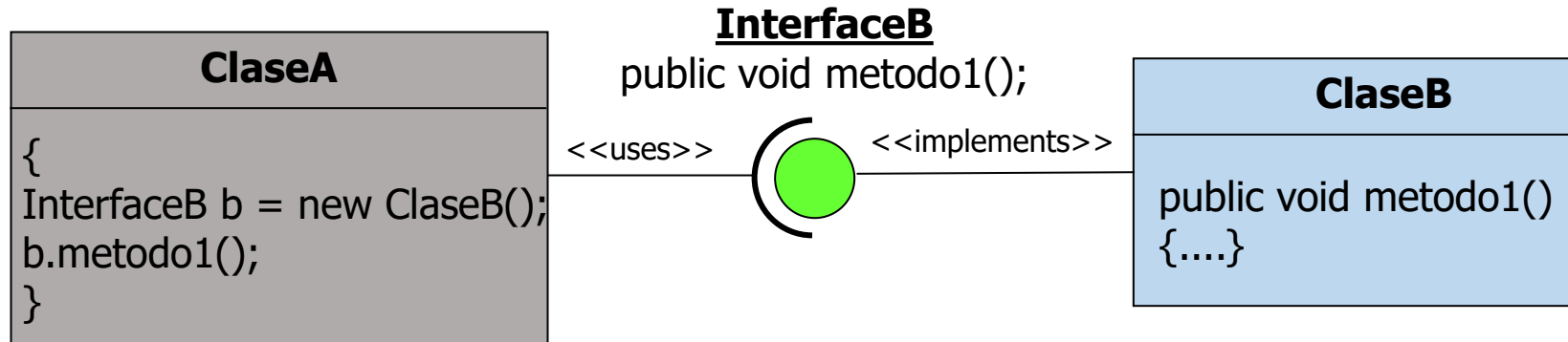


Reducir el Acoplamiento



- Cada capa solamente se comunica con las capas contiguas y por lo tanto solamente depende de ellas.
- Ej: No hay dependencia entre la capa de Presentación y la de Persistencia

Reducir el acoplamiento - Interfaces



Con el uso de interfaces, las referencias empleadas en una parte de la aplicación no quedan acopladas a una clase (implementación concreta), sino a una "interface".

SECCIÓN

Inyección de Dependencias



Reducir el Acoplamiento – Inyección de Dependencias (DI)

ClaseA
<pre>{ InterfaceB b = new ClaseB(); b.metodo1(); }</pre>

DI por parámetro

ClaseA
<pre>InterfaceB b; public void setB(InterfaceB b){ this.b=b; } ... b.metodo1();</pre>

DI por constructor

ClaseA
<pre>InterfaceB b; public ClaseA (InterfaceB b){ this.b=b; } ... b.metodo1();</pre>

Creación de un bean

```
@Configuration
public class A04Config {

    @Bean(name="unaPersona")
    public PersonaInterface getPersona(){
        Persona p = new Persona();
        p.setId(23);
        p.setNombre("Pedrito");
        p.setTelefono("1234546778");
        return p;
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/ ...>

    <bean id="unaPersona" class="agenda.Persona">
        <property name="id" value="23"/>
        <property name="nombre">
            <value>Pedrito</value>
        </property>
        <property name="telefono" value="1234546778"/>
    </bean>

</beans>
```

Inyección de dependencias (DI)

```
@Bean(name="unPadre")
public PersonaInterface padre(){
    Persona p = new Persona();
    p.setNombre("Pepe");
    return p;
}

@Bean(name="unaMadre")
public PersonaInterface madre(){
    Persona p = new Persona();
    p.setNombre("Ana");
    return p;
}
```

```
@Bean(name="unaFamilia")
public Familia getFamilia(){
    Familia f = new Familia();
    f.setPadre(padre());
    f.setMadre(madre());
    return f;
}
```

```
<bean id="unPadre" class="agenda.Persona">
    <property name="nombre" value="Pepe" />
    <property name="telefono" value="1234546778"/>
</bean>

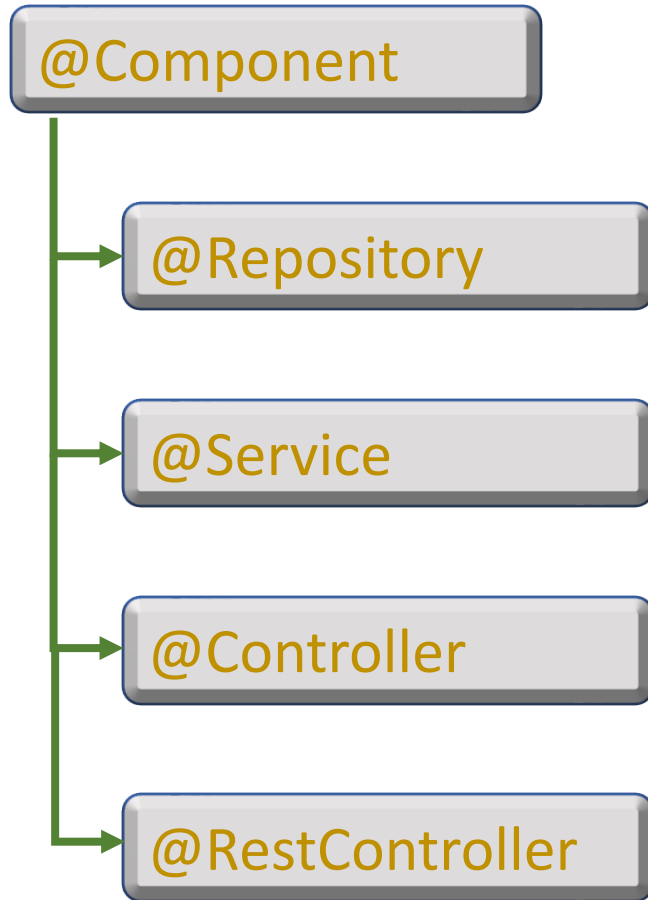
<bean id="unaMadre" class="agenda.Persona">
    <property name="nombre" value="Ana" />
    <property name="telefono" value="987654321"/>
</bean>

<bean id="unaFamilia" class="agenda.Familia">
    <property name="padre" ref="unPadre" />
    <property name="madre" ref="unaMadre" />
</bean>
```

```
@Component
public class Familia {
    private Persona padre;
    private Persona madre;

    @Autowired
    public Familia(Persona padre, Persona madre) {
        this.padre = padre;
        this.madre = madre;
    }
}
```

@Component (stereotype annotations)



```
@Component  
public class MiComponenteGenerico { }
```

```
@Service  
public class UsuarioService { }
```

```
@Repository  
public class UsuarioRepository { }
```

```
@Controller  
public class UsuarioController { }
```

```
@RestController  
public class UsuarioRestController { }
```

SECCIÓN

Spring Boot



SECCIÓN

Soporte a la configuración



Soporte de configuración en Spring Boot

- Spring Boot permite externalizar configuración para usar el mismo código en distintos entornos
 - application.properties
 - application.yml
 - perfiles: dev, test, prod
 - variables de entorno
 - argumentos de línea de comandos
 - configuración por entorno: application-dev.yml, application-prod.yml
 - @ConfigurationProperties



application.properties

spring.application.name=presentaciones

server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/salas

spring.datasource.username=app

spring.datasource.password=secreta

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

app.sales.capacidad-maxima=30

app.sales.permitir-reservas-externas=true

application.yaml

server:

port: 8081

spring:

datasource:

url: jdbc:mysql://localhost:3306/salas

username: app

password: secreta

driver-class-name: com.mysql.cj.jdbc.Driver

application:

name: presentaciones

app:

salas:

capacidad-maxima: 30

permitir-reservas-externas: true

@ConfigurationProperties

```
@ConfigurationProperties(prefix = "app.salas")
public class SalaProperties {

    private Integer capacidadMaxima;
    private Boolean permitirReservasExternas;

    public Integer getCapacidadMaxima() {
        return capacidadMaxima;
    }
    //...
```

```
@ConfigurationPropertiesScan
@SpringBootApplication
public class PresentacionesApplication {

    public static void main(String[] args) {
        SpringApplication.run(PresentacionesApplication.class, args);
    }
}
```

Perfiles

- application.yml
 - Configuración común a la aplicación
- application-dev.yml
 - Configuración de base de datos y otros recursos en entorno de desarrollo
- application-test.yml
 - Configuración de base de datos y otros recursos en entorno de pruebas
- application-prod.yml
 - Configuración de base de datos y otros recursos en entorno de producción

```
PABLO:~$ export SPRING_PROFILES_ACTIVE=prod
```

```
:~$ java -jar app.jar --spring.profiles.active=prod
```



Variables de entorno

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/salas
    username: app
    password: ${DB_PASSWORD}
    driver-class-name: com.mysql.cj.jdbc.Driver
  application:
    name: presentaciones
```

Linux/maxOS

```
~$
~$ export DB_PASSWORD=mi_password_real
```

```
ubuntu@THINKPAD-PABLO:~$
ubuntu@THINKPAD-PABLO:~$ echo $DB_PASSWORD
mi_password_real
```

Windows (CMD)

```
C:\Users\plope>setx DB_PASSWORD "mi_password_real"
CORRECTO: se guardó el valor especificado.
```

```
C:\Users\plope>echo %DB_PASSWORD%
mi_password_real
```

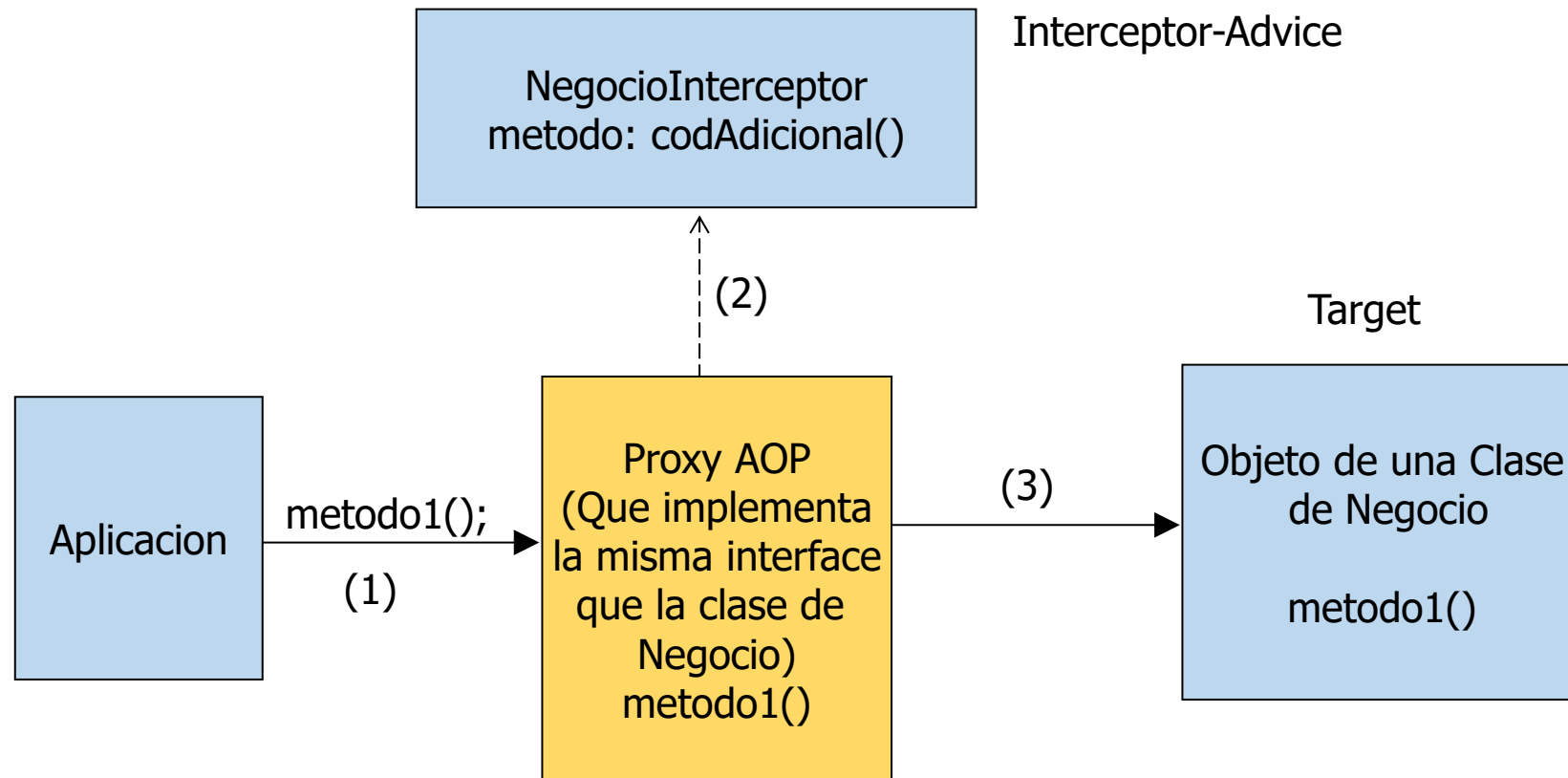
SECCIÓN

Aspect Oriented Programming - AOP



Spring AOP

Cuando la aplicación le solicita al contexto un bean de negocio, el contexto le suministra una referencia al Proxy que implementa la misma interface que el Objeto de Negocio. Cuando la aplicación ejecuta el método de negocio, el proxy llama primero al invoke del interceptor.



AOP - Terminología

Cross-cutting concerns (problema transversal): Determinadas clases de una aplicación comparten requerimientos secundarios comunes.

Por ejemplo, añadir logging a las clases de acceso a los datos y a las clases de negocio cada vez que se ejecuta uno de sus métodos. Aunque la funcionalidad principal de cada clase sea muy diferente, el código necesario para realizar la segunda funcionalidad es casi el mismo.

Advice (Consejo): Este es el código adicional que se quiere aplicar al modelo existente. En el ejemplo, es el código de logging que se quiere ejecutar cada vez que se entre en un método de las clases de datos o negocio.

Point-cut (punto de ruptura o punto de corte): Conjunto de puntos de ejecución de la aplicación (que cumplen un patrón o plantilla) donde se desea aplicar el código adicional de un **Advice**. En el ejemplo, se tendrá un punto de corte en la entrada de los métodos de las clases de acceso a datos y negocio. Cada uno de estos puntos de ejecución se llama **Joinpoint**.

Aspect (Aspecto): La combinación de un punto de corte y un consejo es un aspecto. En el ejemplo, será la aplicación del consejo (advice) de logging a los puntos de corte (point-cut) de los métodos de negocio y acceso a datos.

Tips de Interceptores

Se pueden declarar beans interceptores de 5 tipos, dependiendo de cuándo se ejecutarán.

Antes de invocar al método	<code><aop:before method=""/></code>
Alrededor (Encapsula al método)	<code><aop:around method=""/></code>
Después de invocar al método	<code><aop:after method=""/></code>
Después de finalizar con éxito	<code><aop:after-returning method=""/></code>
Después de lanzar una excepción	<code><aop:after-throwing method=""/></code>

AspectJ pointcut designators (PCD)

execution: para referenciar join points de ejecución del método

within: para referenciar join points dentro de ciertos tipos (clase o paquete.*)

target: join points de un tipo determinado (Clase)

args: join points donde los argumentos son instancias de los tipos dados

@target: join points donde la clase del objeto en ejecución tiene una anotación del tipo dado

@args: join points donde el tipo en tiempo de ejecución de los argumentos reales pasados tiene anotaciones de los tipos dados

@annotation: join points donde el método tiene la anotación dada

SECCIÓN

Acceso a Datos

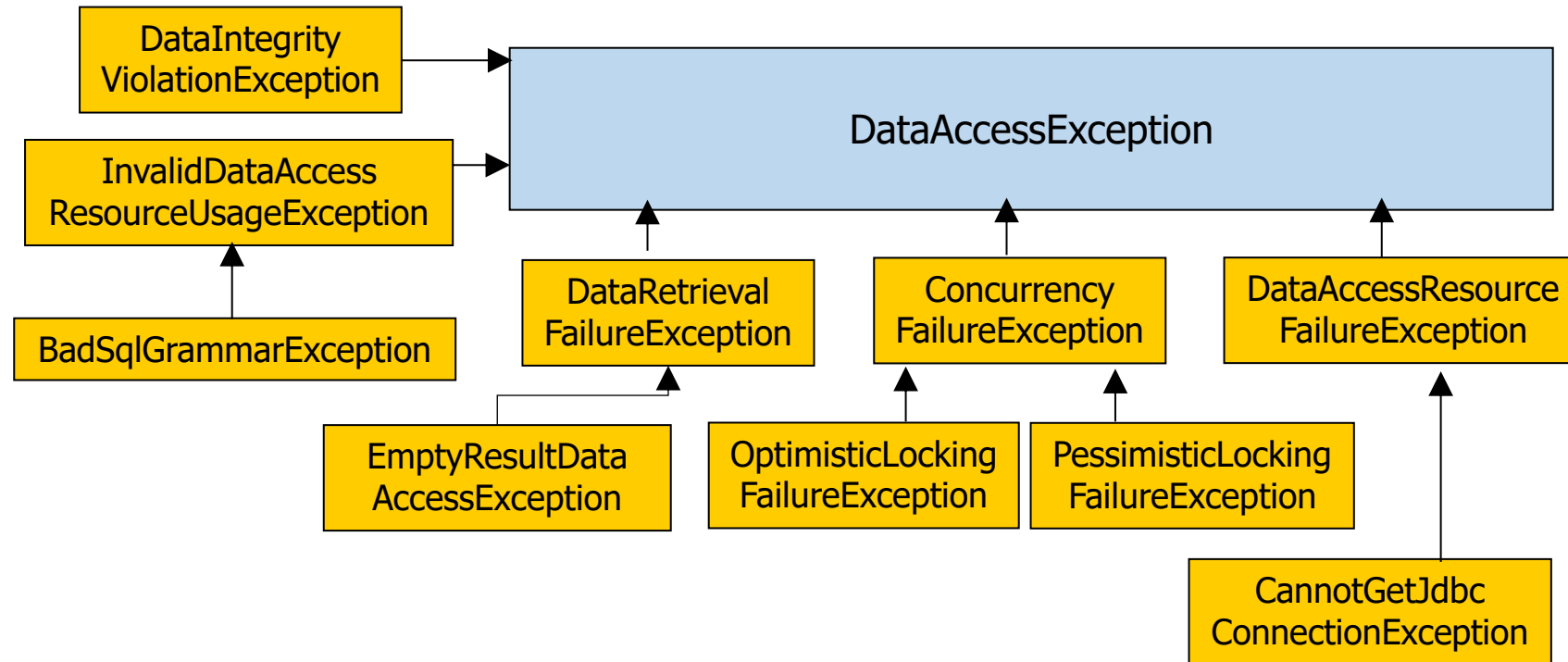


Integración de Spring con persistencia

Permite la implementación del Patrón DAO a partir de Clases específicas para distintas tecnologías de persistencia

- **JDBC**
- **Hibernate**
- **JDO**
- **MyBatis**
- **JPA**

Gestión de Excepciones



DataRetrievalFailure: Error al recuperar los datos desde la bbdd.

DataAccessResourceFailure: Error al conectar con la bbdd.

DataIntegrityViolation: Cuando se ha violado alguna regla de integridad. Ej. clave primaria duplicada.

Transacciones – Características ACID

ÁTÓMICO

Las transacciones están compuestas de una o más actividades agrupadas como una sola unidad de trabajo.

CONSISTENTE

Finalizada una transacción, de forma correcta o no, el sistema debe quedar en un estado consistente.

AISLADO

Las transacciones deben soportar acceso a múltiples usuarios, de forma que las transacciones deben aislarse una de otras y evitar el acceso concurrente.

DURABLE

Una vez finalizada una transacción, el resultado debe persistir y tolerar cualquier tipo de fallo del sistema.

Transacciones - responsabilidades

¿ DÓNDE DEBEN CONTROLARSE LAS
TRANSACCIONES?

¿ EN LOS DAOS ?

**SE DEBEN CONTROLAR A NIVEL DE SERVICIO
LA LÓGICA DE NEGOCIO DEFINE LAS
TRANSACCIONES**

Spring: Gestión declarativa de Transacciones

- Permite especificar declarativamente qué métodos de negocio constituyen una transacción. Mediante proxies de programación orientada a aspectos se maneja el datasource para deshacer la transacción si se detecta alguna excepción.
 - No necesita un servidor de aplicaciones para ejecutarse.
 - No es dependiente de la tecnología de acceso a datos(Jdbc, Hibernate, etc.).
 - Se pueden declarar reglas que indiquen que excepciones provocan un rollback.
 - Se puede declarar la forma en cómo se propagan.

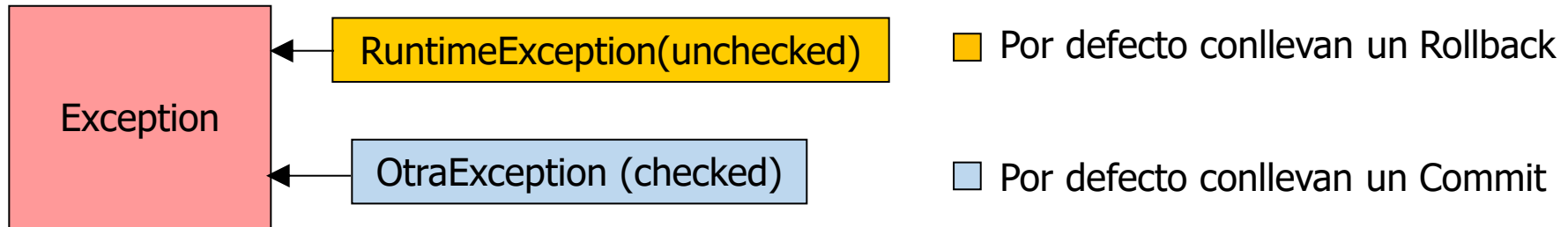
Gestión declarativa de Transacciones

```
@Transactional  
public void metodoTransaccional() {  
    ...  
}
```

- Una tx comienza justo antes de la primera línea del método
- Finaliza justo después de la última línea
- Si el método finaliza con éxito se ejecuta un commit si finaliza con una excepción del tipo RuntimeException realiza un rollback
- Si llama a un método cualquiera que no está anotado con @Transactional, este entra dentro de la transacción

Transacciones - rollback

La excepción `DataAccessException` de Spring es la clase raíz de la jerarquía de excepciones de acceso a datos
`DataAccessException` descende de `RuntimeException`, es decir, es una excepción “**unchecked**”



En la anotación **@Transactional** se puede especificar qué excepciones producen **rollback** y cuales no.

@Transactional (**rollbackFor={`ClienteYaExisteException.class`},**
noRollbackFor={`SaldoNegativoException.class`})

Propagación de transacciones

- **REQUIRED:** El método siempre es transaccional, bien una existente en el contexto desde el que es llamado, o en caso contrario, se le asociará una nueva transacción.
- **REQUIRES_NEW:** El método siempre se ejecuta en una nueva transacción, aunque sea llamado desde un ámbito transaccional.
- **NOT_SUPPORTED:** El método no es transaccional por si mismo ni se incluirá dentro de ninguna transacción. Si es llamado y la tx existe, la pone en suspenso y se retoma al finalizar.
- **SUPPORTS:** Si el método es llamado desde un contexto transaccional, se incluirá en la transacción. En caso contrario será un método no transaccional.
- **MANDATORY:** Solamente puede ser llamado desde un contexto transaccional, de lo contrario produce una excepción.
- **NEVER:** No puede ser llamado desde un ambito transaccional. Si se hace, lanza una excepción.

Propagación de transacciones

Atributo	existe transacción previa	Efecto
REQUIRED	No	El contenedor crea una nueva transacción
	Si	El método se une a la transacción de quien le llama
REQUIRES_NEW	No	El contenedor crea una nueva transacción
	Si	Se crea una nueva transacción y la del llamador se pausa.
SUPPORTS	No	No se usa transacción
	Si	Se suma a la transacción de quien le llama.
MANDATORY	No	Se lanza una Exception
	Si	Se suma a la transacción de quien le llama.
NOT_SUPPORTED	No	No se usa transacción
	Si	No se usa transacción y la transacción de quien le llama se pausa.
NEVER	No	No se usa transacción.
	Si	Lanza una Exception

SECCIÓN

Spring Data



Spring Data - JPA

SpringData genera de forma automática los Daos de las clases persistentes

public interface ClienteRespository extends JpaRepository<Cliente, Integer>{}

Instanciará un ojb que implemente esta interfaz, con 30 métodos:
save(...), findById(...), findAll(...), delete(...), deleteAll(...), exist(...), flush(), etc.

*En lugar de JpaRepository, podemos exponer CrudRepository

Spring Data - JPA

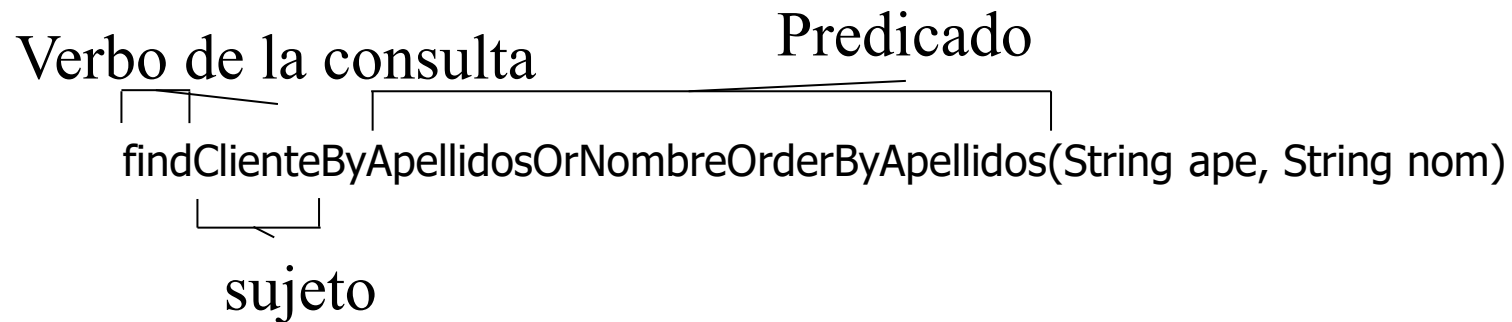
SpringData genera de forma automática los Daos de las clases persistentes.

```
public interface ClienteRespository extends JpaRepository<Cliente, Integer>{  
  
    // Mediante el nombre SpringData construye la consulta  
    tipoRetorno verboSujetoByPredicado(parametros);  
  
    @Query(consulta JPA o consulta SQL nativa)  
    declaración normal de método  
  
}
```

Spring Data - JPA

Extensión de la funcionalidad de JpaRepository

Opción 1: Crea las consultas analizando el nombre del método



Verbo: get read find count (los 3 primeros son sinónimos)

Sujeto: se ignora, excepto si comienza con Distinct

Predicado:

atributoOperador + And/Or + ... + [IgnoringCase] [OrderBy]

```
public List<Cliente> findByNameOrApellidosAllIgnoringCase(String nom, String ape);
```

Spring Data - JPA

Opción 1: Crea las consultas analizando el nombre del método

Palabra Clave	Ejemplo	Clausula where generada
And	findByLastnameAndFirstname	... where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	... where x.lastname = ?1 or x.firstname = ?2
Is, Equals	findByFirstname, findByFirstnames, findByNameEquals	... where x.firstname = ?1
Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age <= ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age >= ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull	findByAgeIsNull	... where x.age is null
IsNotNull, NotNull	findByAge(Is)NotNull	... where x.age not null
Like	findByFirstnameLike	... where x.firstname like ?1

Spring Data - JPA

Opción 1: Crea las consultas analizando el nombre del método

Palabra Clave	Ejemplo	Clausula where generada
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	... where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	... where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
True	findByActiveTrue()	... where x.active = true
False	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstname) = UPPER(?1)

Spring Data - JPA

Opción 2: Consulta JPQL y Nativas con la anotación @Query

```
public interface ClienteRepository extends JpaRepository<Cliente, Integer> {  
  
    @Query("select c from Cliente c where c.apellidos like ?1%")  
    List<Cliente> findByApellidos(String apellido);  
  
    // Parámetros con nombre  
    @Query("select c from Cliente c where c.nombre= :nombre and  
           c.apellidos = :apellidos")  
    List<Cliente> findByNombreCompleto(@Param("apellidos") String ape,  
                                       @Param("nombre") String nom);  
  
    // Query Nativa  
    @Query(value = "SELECT * FROM CLIENTES WHERE EMAIL = ?1",  
           nativeQuery = true)  
    Cliente findByCorreo(String correo);  
}
```

Spring Data - JPA

Opción 3: Crear una clase con métodos implementados. Se “heredarán” en nuestro repositorio

1. Crear una interfaz con la declaración de los métodos, el nombre es irrelevante
2. Crear una clase que implemente esta interfaz con los métodos implementados.
 1. Como atributo, es muy probable que se necesite el EntityManager, irá entonces anotado como `@PersistenceContext`
 2. Podría ser útil tener disponibles los métodos CRUDs para utilizar en el método, para ello incorporamos un atributo de la interfaz del repositorio como `@Autowired`
 3. Su nombre debe ser igual a la interfaz repositorio y un sufijo, por defecto debe ser `Impl`.
 4. Para utilizar otro sufijo, debemos incluir en la configuración:

repository-impl-postfix=“OtroSufijo” en la etiqueta <jpa:repositories>
*<jpa:repositories base-package=“persistencia” /> Busca los finalizados en **Impl***
<jpa:repositories base-package=“persistencia” repository-impl-postfix=“PLG”/>

3. La interfaz repositorio, extenderá de `JpaRepository` y de la interfaz nueva

Spring Data - JPA

Opción 3: Crear una clase con métodos implementados. Se “heredarán” en nuestro repositorio

```
public interface ClienteRepositoryCustomizado {  
    public void miMetodo(Cliente c); }  
  
Public class ClienteRepositoryImpl implements ClienteRepositoryCustomizado {  
    @PersistenceContext  
    private EntityManager em; // no es obligatorio  
    @Autowired  
    private ClienteRepository dao; // no es obligatorio  
  
    public void miMetodo(Cliente c){  
        //Codigo del método  
    }  
  
    public interface ClienteRepository extends JpaRepository<Cliente, Integer>,  
        ClienteRepositoryCustomizado {  
        ...  
    }  
}
```